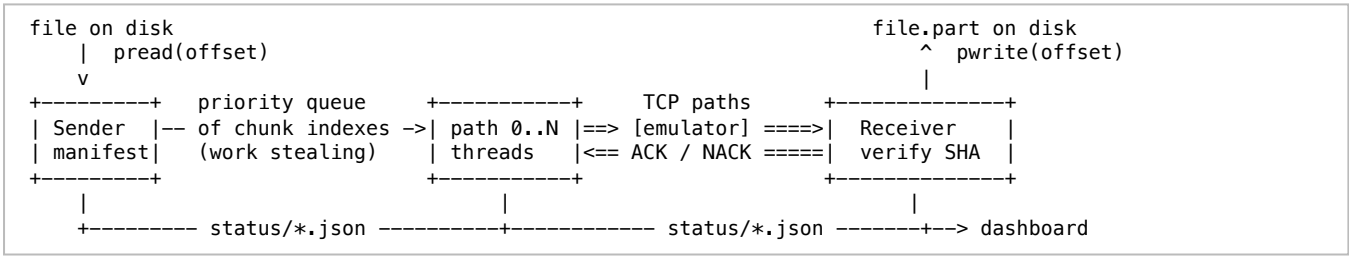
Architecture

Components

Component	File	Role
Protocol	`mpft/protocol.py`	Header pack/unpack, CRC, `send_msg`/`recv_msg`, manifest builder
Sender	`mpft/sender.py`	One thread per path, shared chunk queue, retries, recovery metrics
Receiver	`mpft/receiver.py`	Accepts connections, verifies chunks, writes at offsets, final verify and rename
Emulator	`mpft/emulator.py`	TCP proxy per path: delay, jitter, rate limit, kill, corruption
Status	`mpft/status.py`	Writes `status/<role>-<session>.json` once a second
Metrics	`mpft/metrics.py`	Builds the result dict of a transfer
CLI	`mpft/cli.py`	`send`, `receive`, `emulate`, `dashboard`
Dashboard	`web/`	FastAPI app that reads the status files
Experiments	`experiments/`	Automated scenarios and plots

Data flow



The sender hashes the file once in a streaming pass (per-chunk hashes and the whole-file hash go into the manifest), then reads each chunk with `os.pread` only when a path takes it. The receiver writes each verified chunk with `os.pwrite` at `index * chunk_size`. Neither side holds the whole file in memory: at most one chunk per path is in RAM.

Scheduler (work stealing)

```
queue = PriorityQueue()              # items (priority, chunk_index)
for i in chunks the receiver does not hold:  queue.put((1, i))

path_worker(p):                       # runs in its own thread
    while not finished:
        if no connection: backoff, connect, (first path only) exchange MANIFEST
        (_, i) = queue.get(timeout=0.1)
        send CHUNK i; wait for reply (timeout 5 s)
        ACK            -> record bytes and RTT
        NACK           -> requeue(i)
        timeout/reset  -> requeue(i), close socket, reconnect later

requeue(i):
    attempts[i] += 1
    if attempts[i] > 5: fail the transfer
    after min(0.05 * 2^(attempts-1), 1.0) s: queue.put((0, i))    # priority 0 = before fresh chunks
```

A fast path comes back to the queue sooner, so it takes more chunks. No bandwidth estimation is needed. Measured: with paths of 80/20/5 Mbit/s the byte shares were 74 % / 20 % / 6 % (TEST_REPORT E2).

Trade-offs

Chunk size. Small chunks balance load better and waste less when a chunk is lost, but cost more headers, hashes and round trips (each path waits for an ACK per chunk). Large chunks use the link better but make the load balance coarse at the end of a transfer and make a retransmit expensive. Measured with 4 paths: 64 KiB gave 50 Mbit/s, 256 KiB 90, 1 MiB 107, 4 MiB 107 (E6). Default 1 MiB; experiments use 256 KiB.

Work stealing vs round robin. Round robin gives every path the same number of chunks, so the slowest path finishes last and the fast paths sit idle. Work stealing lets idle paths take the remaining chunks, and a dead path simply stops taking work. The cost is a shared queue with a lock, which is negligible here.

TCP per path vs UDP. TCP gives ordered, reliable bytes, congestion control and works through NAT and firewalls, so MPFT only needs framing and chunk-level recovery. A UDP design would need its own loss detection, retransmission and congestion control, and does not suit the "survive a dead path" demo as simply. The price: TCP head-of-line blocking inside one path, and one chunk in flight per path (stop-and-wait), which is why one 40 Mbit/s emulated path reaches only about 23 Mbit/s (E1). More paths hide this latency.

SHA-256 plus CRC-32. CRC-32 on the 16 header bytes is cheap and catches a damaged header so the receiver never trusts a wrong length. SHA-256 per chunk catches data corruption, identifies exactly which chunk to resend and is what resume uses to trust bytes already on disk. SHA-256 of the whole file proves the final result. CRC-32 is not cryptographic, so it is used only for framing.

Single-process, thread-per-path. Simple to explain and debug. Python's GIL limits CPU-bound speed, but the workload is mostly waiting on sockets and the 8-path experiment reached 172 Mbit/s.

ACK per chunk, not per byte. Application-level ACKs make "stored and verified" explicit, which TCP's ACK cannot say.